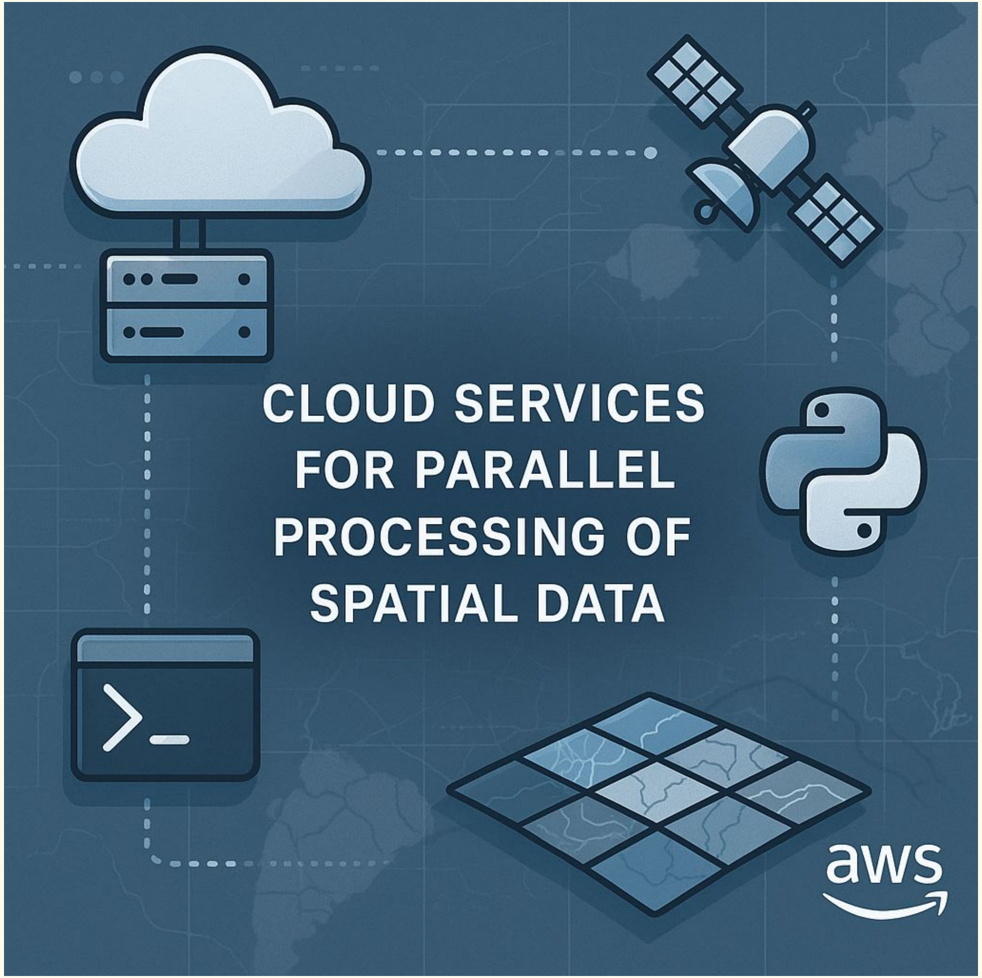


A. John Woodill
Research Associate
Oregon State University
johnwoodill@gmail.com

October 27, 2025

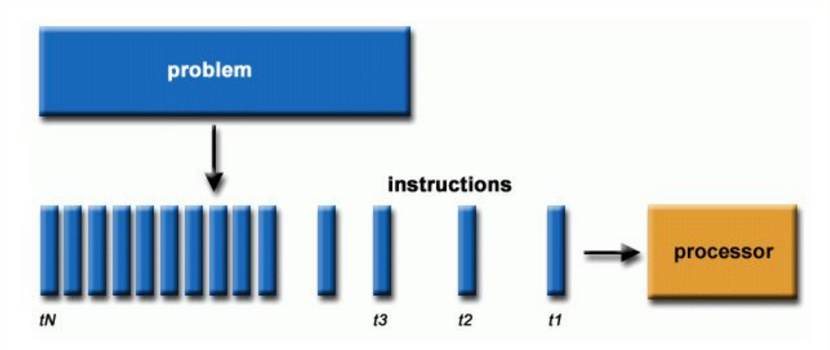
Lecture Repo:
`https://github.com/johnwoodill/parallel-processing-demo`

Why Cloud for Spatial Data?

- Spatial data is massive: rasters, shapefiles, time series, remote sensing.
- Local machines choke on large jobs (limited RAM and CPU/GPU).
- Cloud lets you scale compute to the problem.
- **Parallelization** reduces runtime from hours to minutes
    - Serial: 3 Hours
    - Parallel: 31 minutes

Serial vs Parallel Processing

Serial
(for loop)

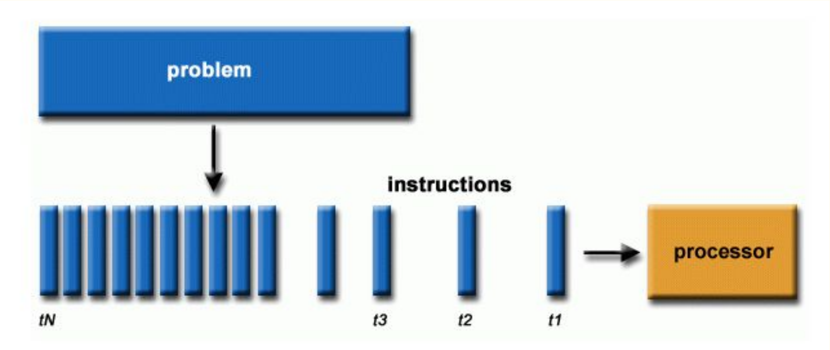


One grid at a time

Serial vs Parallel Processing

Serial

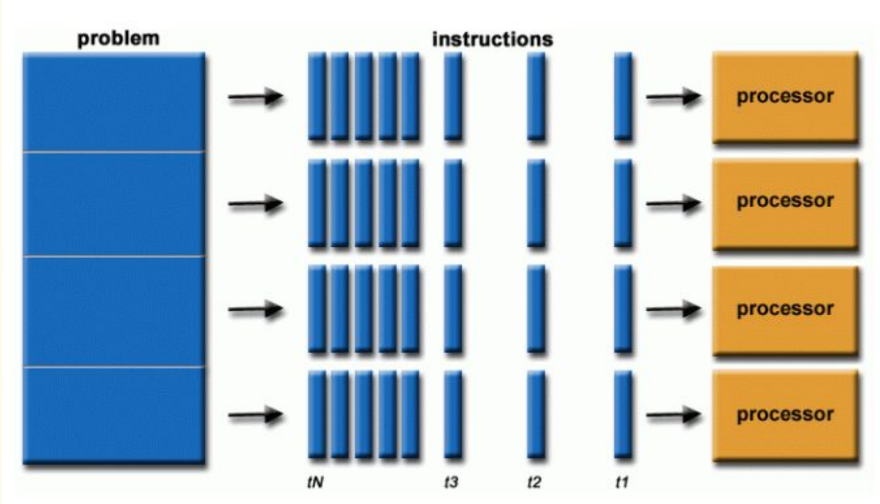
(for loop)



One grid at a time

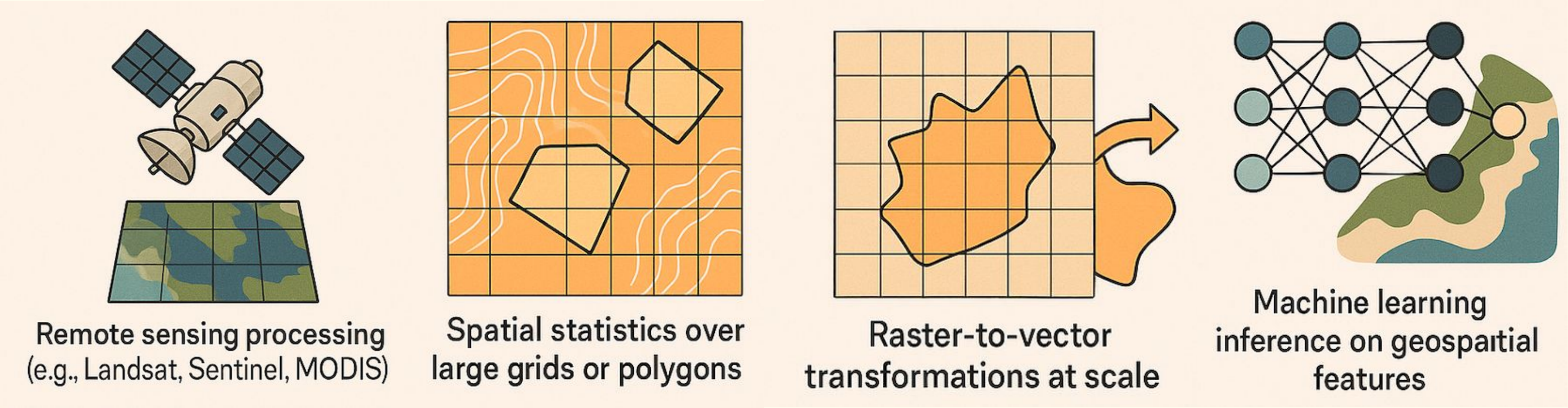
Parallel

(dot product)



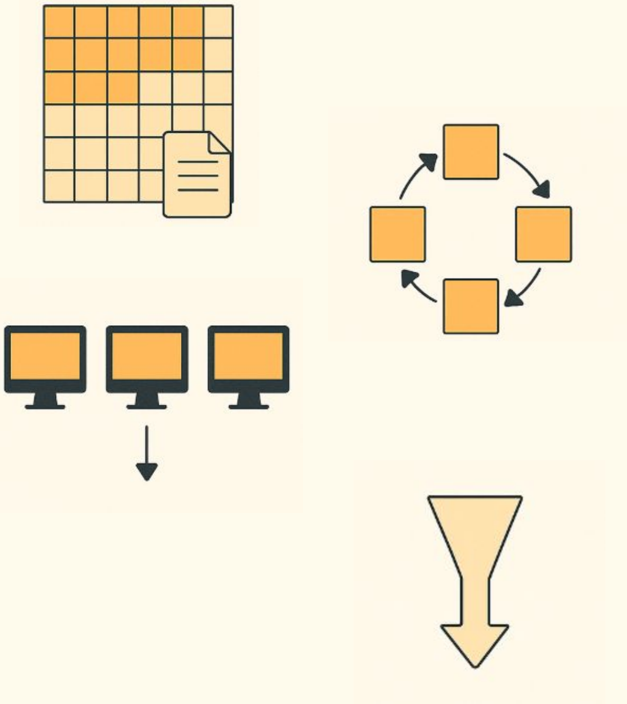
Many grids at a time

Parallel Use Cases



Parallelization Strategy

- Split spatial domain or file list into chunks
- Assign each chunk to a process or worker
- Run computations simultaneously
- Write outputs to S3 or local storage
- Aggregate results downstream



Development Tools

Local Computer

- **Local dev:** GitHub for version control
- **Access:** SSH into remote instance
- **Runtime:** EC2 with tmux for long-running jobs
- **Package management:** pip/R in virtual env
- **Parallelization:** native Python or libraries like multiprocessing, Dask, or joblib.

Project Overview

1. Launch EC2 instance (e.g., t3.medium or c5.4xlarge depending on job
2. Setup Github keys
3. SSH into EC2 instance
4. Clone repo and setup dependencies (Python, R)
5. Execute parallel processing job
6. Monitor, debug, and collect output

Live Demo

Other Considerations

- GPU/CPU
- Other services: runpod.io, vast.ai, Azure, Google Cloud
- Packages (Dask, Ray, etc)
- Serverless vs EC2
- OSU HPC (Cluster - submitting jobs)

Questions?